

# Verifiable Cross-Modal Searchable Encryption via Hierarchical Spherical Tree with Beam Search

Yuzhe Wang

**Abstract**—Cross-modal searchable encryption enables secure and efficient retrieval of multimedia data (e.g., images and texts) over encrypted cloud data. However, existing schemes face challenges in achieving both high search accuracy and efficiency, particularly on large-scale datasets. To address these problems, we propose a novel Verifiable Cross-Modal Searchable Encryption scheme based on Hierarchical Spherical Tree with Beam Search (VCSE-HST). First, we construct a hierarchical  $k$ -ary spherical tree index structure using recursive spherical  $k$ -means clustering, which significantly reduces tree depth. Second, we introduce a beam search algorithm that maintains multiple candidate paths during traversal, avoiding the local optimum problem of greedy search. Third, we design a lightweight score correctness verification mechanism under encryption and further add a *Merkle-tree-based integrity verification* that enforces path completeness and result inclusion with negligible overhead. Security analysis shows that our scheme protects the privacy of indexes, trapdoors, and queries while providing verifiable score correctness and execution integrity. Extensive experiments on CIFAR-100 (50,000 images) and Caltech256 (30,607 images) demonstrate that VCSE-HST achieves 90% Recall@1 (matching state-of-the-art LSH-based schemes) while being  $7.5\times$  faster in search time at beam=10. With beam size tuning, our scheme achieves up to  $19.7\times$  speedup over the LSH baseline with only 7% accuracy loss; Merkle verification adds only a few milliseconds per query and grows near-linearly with the beam size, enabling practical verifiability.

**Index Terms**—Cross-modal retrieval, searchable encryption, hierarchical spherical tree, beam search, integrity verification, Merkle tree, cloud computing

## I. INTRODUCTION

WITH the rapid growth of multimedia data in cloud computing, cross-modal retrieval has become increasingly important for applications such as content-based image retrieval, social media search, and multimodal recommendation systems [?]. However, privacy concerns arise when outsourcing sensitive multimedia data to untrusted cloud servers. Cross-modal searchable encryption (CMSE) techniques allow users to search over encrypted multimodal data without revealing data content or query information to the cloud server.

Existing CMSE schemes can be categorized into two main approaches: locality-sensitive hashing (LSH) based methods [?], [?] and tree-based indexing methods [?], [?]. LSH-based methods achieve sublinear search complexity by hashing similar items into the same buckets; while they can attain high recall rates (e.g., 90% Recall@1 in [?]), this requires maintaining many hash tables (e.g., 64 tables), leading to

substantial memory overhead and increased query latency from probing multiple tables. Tree-based methods organize encrypted data in hierarchical structures (e.g., binary trees) to enable efficient pruning during search, but existing binary tree approaches have several limitations: (1) **Deep tree structure**: Binary trees require  $O(\log_2 n)$  depth for  $n$  documents, leading to accumulated errors during greedy traversal. (2) **Coarse semantic partitioning**: Each node only splits data into two branches, resulting in weak semantic clustering. (3) **Local optimum problem**: Greedy search strategies easily fall into local optima, causing significant accuracy loss.

Beyond accuracy and efficiency, **verifiability** is essential in outsourced search scenarios. Cloud servers may not only be curious about user data but also attempt to reduce computational costs by executing incomplete searches—skipping expensive tree traversals, returning local optima, or omitting required operations—yet still charge full service fees. This motivates the need for mechanisms that enable users to verify both the correctness of returned scores and the integrity of the search execution.

To address these challenges, we propose a novel Verifiable Cross-Modal Searchable Encryption scheme based on Hierarchical Spherical Tree with Beam Search (VCSE-HST). Our main contributions are summarized as follows:

- We propose a **hierarchical  $k$ -ary spherical tree** index structure using recursive spherical  $k$ -means clustering. Our structure achieves shallow depth (3–4 layers on CIFAR-100/Caltech256) while providing strong semantic partitioning with  $k = 10$  branches per node.
- We introduce a **beam search algorithm** for encrypted tree traversal, which maintains top- $\beta$  candidate paths at each level instead of a single greedy path. This global pruning strategy effectively avoids local optima and provides flexible trade-offs between accuracy and efficiency by tuning the beam size  $\beta$ .
- We design a lightweight **score correctness verification** mechanism under encryption that allows users to verify the correctness of returned inner-product scores, ensuring that the reported similarity values are computed accurately from the encrypted vectors.
- We further realize **Merkle-tree-based integrity verification** for beam search, committing to the encrypted index and returning layer-by-layer inclusion proofs that enforce correct node expansions and global top- $\beta$  pruning. This provides execution integrity (path completeness and result inclusion), complementing score correctness with only few-millisecond overhead per query.
- We incorporate a **variance-guided dimension reduction**

Y. Wang is with the School of Computer Science and Technology, East China Normal University, Shanghai 200062, China. E-mail: yzwang@stu.ecnu.edu.cn

module as a core preprocessing component, with an *encrypted rule* (AES-256) to consistently prune low-variance dimensions on both index and query sides. The module can be disabled (no compression, used in our main results) or tuned to achieve memory/communication savings; interestingly, modest compression sometimes improves Recall@1 by removing low-discriminative dimensions. Sec. ?? reports detailed trade-offs.

- We conduct experiments on CIFAR-100 (50,000 images, 100 text queries) and Caltech256 (30,607 images, 257 text queries), comparing our scheme with LSH-based Cross-Model-SE [?] and linear search. VCSE-HST with beam size 10 achieves 90% Recall@1 (matching LSH) while being  $7.5\times$  faster; with beam size 3, it achieves up to  $19.7\times$  speedup with only 7% R@1 loss.

The rest of this paper is organized as follows: Section ?? reviews related work on searchable encryption and cross-modal retrieval. Section ?? presents the system model, problem definition, and threat model. Section ?? describes the construction of VCSE-HST. Section ?? provides security analysis. Section ?? demonstrates experimental results. Section ?? concludes this paper.

## II. RELATED WORK

### A. Searchable Encryption

Searchable symmetric encryption (SSE) enables users to search over encrypted data without decryption. Song et al. [?] proposed the first practical SSE scheme supporting single keyword search. To improve search expressiveness, ranked keyword search schemes [?], [?] were proposed to return top- $k$  most relevant results rather than all matching results.

Miao et al. [?] presented a machine learning-based ranked keyword search scheme (ML-RKS) using  $k$ -means clustering and balanced binary trees to reduce search complexity. Their scheme achieves sublinear search time  $O(mz \log L_y)$  where  $m$  is the number of keywords,  $z$  is the number of matching leaf nodes, and  $L_y$  is the cluster size. However, their binary tree structure still suffers from deep hierarchies and coarse semantic partitioning. Our VCSE-HST improves upon this by using  $k$ -ary trees ( $k = 10$ ) instead of binary trees, achieving much shallower depth and finer-grained semantic clustering.

Li et al. [?] proposed VRFMS, a verifiable ranked fuzzy multi-keyword search scheme that uses locality-sensitive hashing and bloom filters for fuzzy matching, and employs homomorphic MAC for verification. While VRFMS provides score correctness verification similar to our approach, it does not address the execution integrity of the search algorithm itself. Our scheme complements score verification with Merkle-tree-based integrity verification that enforces path completeness and result inclusion during beam search.

Beyond basic keyword search, researchers have also explored lightweight fine-grained search [?] and attribute-based multi-keyword search [?] for fog computing and mobile crowdsourcing scenarios. Dynamic searchable encryption with forward security has been investigated for medical cloud data [?] and e-healthcare applications [?]. While our current work focuses on static settings, the hierarchical tree structure

can potentially be extended to support dynamic updates with forward security, which we leave for future work.

### B. Cross-Modal Retrieval

Cross-modal retrieval aims to retrieve semantically similar items across different modalities (e.g., image-text, audio-video). Deep learning methods, particularly CLIP (Contrastive Language-Image Pre-training) [?], have achieved remarkable success by learning joint embedding spaces where semantically similar items from different modalities are close together. CLIP uses a contrastive learning approach to align visual and textual representations, enabling zero-shot cross-modal retrieval without task-specific training.

### C. Privacy-Preserving Cross-Modal Retrieval

Combining searchable encryption with cross-modal retrieval, several privacy-preserving cross-modal retrieval (PPCMR) schemes have been proposed. Early work focused on privacy-preserving content-based image retrieval using encrypted features [?], [?], with schemes like BOEW [?] using bag-of-encrypted-words for cloud-based image search. For medical imaging scenarios, specialized schemes [?] have been developed to enable fast nearest neighbor search over encrypted medical images.

Wang et al. [?] designed a two-branch feature extraction network on encrypted data and used cosine distance for similarity measurement. Zhang et al. [?] proposed PITR, which freezes pre-trained model parameters and incorporates trainable adapters to enhance zero-shot performance while using asymmetric scalar-product-preserving encryption. Yang et al. [?] presented MU-TEIR for encrypted image retrieval in multi-user settings with watermark-based traceability. To support flexible query scenarios, Wang et al. [?] proposed an outsourced image retrieval framework that supports unshared keys among multiple users.

Most recently, Yang et al. [?] proposed FECMR, a quantum-resistant PPCMR scheme using function-hiding inner product functional encryption and cross-modal hashing. They designed a KM-tree index based on  $K$ -modes clustering to achieve sublinear search complexity. However, their KM-tree is designed for discrete categorical features, not continuous high-dimensional embeddings like CLIP features.

Despite these advances, existing PPCMR schemes face two main challenges: (1) LSH-based methods (e.g., Cross-Model-SE) achieve fast search but may suffer from accuracy degradation due to hash approximations. (2) Tree-based methods achieve good accuracy but incur high search overhead due to deep tree structures. Our VCSE-HST addresses both challenges by combining shallow hierarchical trees with beam search, achieving both high accuracy (90% Recall@1) and significant speedup ( $7.5\times$  faster than LSH-based methods).

## III. PROBLEM FORMULATION

In this section, we present the system model, problem definition, and threat model.

Fig. 1: System model of VCSE-HST. DO builds encrypted hierarchical spherical tree indexes and uploads to CS. DUs obtain keys from DO and submit encrypted queries to CS, which returns top- $k$  encrypted results. DUs verify the correctness of results.

#### A. System Model

As depicted in Fig. ??, our system consists of three entities: Data Owner (DO), Cloud Server (CS), and Data Users (DUs).

- **Data Owner (DO):** DO is responsible for generating secret keys, extracting features from multimedia data using pre-trained models (e.g., CLIP), building encrypted hierarchical spherical tree indexes, encrypting data, and uploading encrypted data and indexes to CS. In dynamic settings, DO can perform update operations.
- **Data Users (DUs):** DUs are authorized users who can search over encrypted data. After obtaining secret keys from DO, DUs generate encrypted query tokens and submit them to CS. Upon receiving encrypted results, DUs decrypt them and verify result correctness.
- **Cloud Server (CS):** CS stores encrypted data and indexes, executes search operations over encrypted data using the hierarchical tree and beam search algorithm, and returns top- $k$  encrypted results to DUs.

Before outsourcing data, the Data Owner (DO) extracts feature vectors from multimedia files using CLIP, normalizes them to the unit sphere, and builds a hierarchical  $k$ -ary spherical tree index using recursive  $k$ -means clustering. Each node in the tree stores an encrypted spherical center vector. DO then encrypts all tree nodes and data files before uploading them to the Cloud Server (CS) (①BuildIndex). When a Data User (DU) wants to search, he/she first generates a trapdoor locally (②GenTrapdoor), then submits the encrypted query trapdoor to CS (③Search). CS performs beam search on the encrypted tree, computes similarity scores in the encrypted domain, and returns top- $k$  encrypted results to DU. Finally, DU verifies the correctness and completeness of the returned results (④Verify).

#### B. Problem Definition

Let  $\mathcal{F} = \{f_1, f_2, \dots, f_n\}$  be a set of  $n$  multimedia files. Using a pre-trained cross-modal model (e.g., CLIP), we extract a  $d$ -dimensional feature vector  $\mathbf{v}_i \in \mathbb{R}^d$  for each file  $f_i$ . We normalize each vector to the unit sphere:  $\mathbf{d}_i = \mathbf{v}_i / \|\mathbf{v}_i\|_2$ , so that  $\|\mathbf{d}_i\|_2 = 1$ .

For a query  $q$  (which can be text or image), we similarly extract and normalize its feature vector  $\mathbf{q} \in \mathbb{R}^d$  with  $\|\mathbf{q}\|_2 = 1$ . The relevance score between a file  $f_i$  and query  $q$  is measured by the inner product (equivalently, cosine similarity for normalized vectors):

$$\mathcal{S}(\mathbf{d}_i, \mathbf{q}) = \mathbf{d}_i \cdot \mathbf{q} = \sum_{j=1}^d d_{i,j} \cdot q_j \quad (1)$$

The goal of cross-modal searchable encryption is to find the top- $k$  files with the highest relevance scores over encrypted

data, without revealing file content, query information, or relevance scores to CS.

**Efficiency Metric:** For  $n$  files, the search complexity of linear search is  $O(n)$ . Tree-based methods aim to achieve sublinear complexity  $O(\log n)$  or better through hierarchical pruning.

**Accuracy Metric:** We use Recall@K to measure accuracy:

$$\text{Recall@K} = \frac{\# \text{ queries where top-K contains target class}}{\# \text{ total queries}} \quad (2)$$

In evaluation, we also report **Agree@1**, defined as the fraction of queries where the top-1 result returned by VCSE-HST matches the top-1 returned by the Cross-Model-SE baseline (used to quantify fidelity across beam sizes).

#### C. Threat Model

Following existing SSE schemes [?], [?], we consider DO and DUs as fully trusted entities, while CS may be adversarial in two ways. Specifically:

- **Adversarial CS:** CS may exhibit two types of adversarial behavior: (1) *Confidentiality attacks* (honest-but-curious model): attempting to infer sensitive information from encrypted data, indexes, queries, and intermediate computation results; (2) *Integrity attacks* (malicious model): reducing computational costs by executing incomplete searches (e.g., skipping expensive tree traversals, returning local optima, omitting required beam expansions, or forging result scores) while still reporting results to reduce operational overhead. CS has access to: (1) encrypted file vectors  $\{\hat{\mathbf{d}}_i\}$ , (2) encrypted tree nodes  $\{\hat{\mathbf{c}}_j\}$ , (3) encrypted query tokens  $\hat{\mathbf{q}}$ , and (4) encrypted similarity scores  $\mathcal{S}(\hat{\mathbf{d}}_i, \hat{\mathbf{q}})$ . Our scheme addresses confidentiality via encryption and integrity via verifiable computation mechanisms.
- **Known Ciphertext Model:** CS only has access to encrypted data and encrypted queries, without additional background knowledge.
- **Known Background Model:** CS may have statistical information about the dataset (e.g., frequency distributions) and may attempt correlation attacks.

**Leakage scope:** Like most practical SSE schemes, we do not hide access patterns (which documents are returned) or search patterns (which tree nodes are visited during beam search). Protecting such patterns requires orthogonal techniques like ORAM [?] or distributed point functions [?] that incur substantial performance overhead; our focus is on achieving strong confidentiality and verifiable integrity within the standard leakage profile of tree-based SSE.

## IV. PRELIMINARIES

#### A. Bilinear Verification for Encrypted Inner Products

In verifiable searchable encryption, two complementary mechanisms ensure result trustworthiness: *correctness* (scores are computed accurately) and *completeness* (all required operations are executed). We first introduce the bilinear verification technique that underpins our score correctness verification.

Let  $\mathbf{E} \in \mathbb{R}^{2\ell}$  denote an encrypted document vector and  $\mathbf{T} \in \mathbb{R}^{2\ell}$  a query trapdoor. The server reports an inner-product score  $s = \langle \mathbf{T}, \mathbf{E} \rangle$ . To enable verification without revealing plaintext, we augment the encryption with authentication vectors: for each document, a random secret tag  $\mathbf{t} \in \mathbb{R}^{2\ell}$  and a secret verification parameter  $\alpha > 0$  (shared between DO and DUs via key distribution, unknown to CS) define an authentication vector  $\mathbf{E}^{\text{auth}}$  via

$$\alpha \mathbf{E}^{\text{auth}} + \mathbf{E} = \mathbf{t}. \quad (3)$$

Similarly, at query time, a random tag  $\mathbf{r} \in \mathbb{R}^{2\ell}$  yields  $\mathbf{T}^{\text{auth}}$  satisfying

$$\alpha \mathbf{T}^{\text{auth}} + \mathbf{T} = \mathbf{r}. \quad (4)$$

The server computes  $c_1 = \langle \mathbf{T}, \mathbf{E}^{\text{auth}} \rangle + \langle \mathbf{T}^{\text{auth}}, \mathbf{E} \rangle$  and  $c_2 = \langle \mathbf{T}^{\text{auth}}, \mathbf{E}^{\text{auth}} \rangle$  without knowledge of  $\mathbf{t}$  or  $\mathbf{r}$ , and returns  $(s, c_1, c_2)$ . The client then verifies the bilinear identity

$$\langle \mathbf{r}, \mathbf{t} \rangle = \alpha^2 c_2 + \alpha c_1 + s. \quad (5)$$

This relation holds if and only if  $s$  is the correct inner product; any tampering violates the identity with overwhelming probability (assuming the server does not know  $\mathbf{t}$ ,  $\mathbf{r}$ , or  $\alpha$ ). This mechanism guarantees *score correctness* for returned items.

### B. Merkle Trees

A Merkle tree [?] is a cryptographic commitment scheme for authenticated data structures. Given a rooted tree  $\mathcal{T}$  and a collision-resistant hash function  $H(\cdot)$ , each node  $u$  is assigned a hash value computed recursively:

**Standard Merkle Tree Construction.** For a leaf node  $u$  storing data  $\text{data}(u)$ :

$$H(u) = \text{Hash}(\text{data}(u)).$$

For an internal node  $u$  with children  $v_1, v_2, \dots, v_t$ :

$$H(u) = \text{Hash}(H(v_1) \| H(v_2) \| \dots \| H(v_t)),$$

where  $\|$  denotes concatenation. The root hash  $H(\text{root})$  serves as a compact commitment to the entire tree. To verify that a node  $u$  is included in the tree, the prover provides a *Merkle proof*: the sibling hashes along the path from  $u$  to the root. The verifier recomputes hashes bottom-up and checks equality with the committed root  $H(\text{root})$ .

**Application to Our Hierarchical Index.** We apply Merkle trees to commit to our encrypted hierarchical spherical tree index. Each node  $u$  carries metadata (encrypted center  $\hat{\mathbf{c}}_u$ , node ID  $\text{id}(u)$ , and for leaves, document IDs  $\text{Docs}(u)$ ). We compute node hashes by incorporating this metadata:

For a leaf node  $u$ :

$$H(u) = \text{Hash}(\text{id}(u) \| \hat{\mathbf{c}}_u \| \text{Docs}(u) \| \text{LEAF}).$$

For an internal node  $u$  with children  $v_1, \dots, v_t$ :

$$H(u) = \text{Hash}(\text{id}(u) \| \hat{\mathbf{c}}_u \| H(v_1) \| \dots \| H(v_t) \| \text{INTERNAL}).$$

The Data Owner publishes the Merkle root  $H(\text{root})$ . During search, the Cloud Server returns inclusion proofs for visited nodes. Under collision resistance, any tampering with the tree structure, node metadata, or search results will be detected with overwhelming probability.

### Algorithm 1 Spherical $k$ -Means

---

**Require:** Normalized vectors  $\{\mathbf{d}_i\}_{i=1}^m$ , branch factor  $k$   
**Ensure:** Centers  $\{\mathbf{c}_j\}_{j=1}^k$  and assignments

- 1: Initialize centers  $\mathbf{c}_1, \dots, \mathbf{c}_k$  by sampling from  $\{\mathbf{d}_i\}$
- 2: **repeat**
- 3:   Assignment:  $\text{cluster}(i) \leftarrow \arg \max_j \mathbf{d}_i^\top \mathbf{c}_j$  for all  $i$
- 4:   Update:  $\mathbf{c}_j \leftarrow \frac{\sum_{i: \text{cluster}(i)=j} \mathbf{d}_i}{\|\sum_{i: \text{cluster}(i)=j} \mathbf{d}_i\|_2}$  for all  $j$
- 5: **until** convergence

---

## V. PROPOSED SCHEME

In this section, we present the construction of VCSE-HST, including key generation, hierarchical spherical tree construction, index encryption, query token generation, beam search algorithm, and two complementary verification mechanisms. Our scheme provides *score correctness verification* (ensuring returned similarity scores are computed accurately) via bilinear authentication vectors introduced in Preliminaries, and *execution integrity verification* (ensuring the beam search algorithm is executed completely and correctly) via Merkle tree commitments. These two mechanisms work together to provide comprehensive verifiability: score correctness guarantees the mathematical accuracy of reported values, while execution integrity prevents the server from omitting required operations or fabricating non-existent results.

### A. Key Generation

Given a security parameter  $\kappa$ , DO generates a secret key

$$SK = (\mathbf{s}, \mathbf{M}, \mathbf{M}^{-1}, \alpha, K_{\text{rule}}),$$

where  $\mathbf{s} \in \{0, 1\}^\ell$  is a random splitting vector for the padded dimension  $\ell$ ,  $\mathbf{M} \in \mathbb{R}^{2\ell \times 2\ell}$  is a random invertible matrix (we use a symmetric construction in implementation),  $\alpha > 0$  is a secret verification parameter (kept from the server, shared only with authorized users), and  $K_{\text{rule}}$  is an AES key used to encrypt the optional compression rule. This key setup follows an ASPE-style inner-product preserving transformation [?], which enables secure similarity computation over encrypted vectors while supporting efficient key management [?].

### B. Hierarchical Spherical Tree Construction

We propose a hierarchical  $k$ -ary spherical tree structure that achieves shallow depth and finer-grained semantic clustering.

1) *Spherical  $k$ -Means Clustering:* Given a set of normalized vectors  $\{\mathbf{d}_1, \dots, \mathbf{d}_m\}$  on the unit sphere, we use the following procedure.

The key difference from Euclidean  $k$ -means is the normalization in updates so centers remain on the unit sphere.

2) *Recursive Tree Construction:* We build the hierarchical  $k$ -ary tree recursively as follows.

On the full datasets, setting  $k = 10$  and  $s_{\min} = 20$  results in a tree with only 3–4 levels. Fig. ?? provides a schematic view of the spherical-tree topology on the unit sphere: the hierarchical structure induces a nested partitioning where each

**Algorithm 2** Build Hierarchical Spherical Tree

---

**Require:** Vectors  $\{\mathbf{d}_1, \dots, \mathbf{d}_m\}$ , branch factor  $k$ , min cluster size  $s_{\min}$ , max depth  $h_{\max}$

**Ensure:** Root node of the hierarchical tree

- 1: Compute center  $\mathbf{c} \leftarrow \frac{\sum_{i=1}^m \mathbf{d}_i}{\|\sum_{i=1}^m \mathbf{d}_i\|_2}$
- 2: **if**  $m \leq s_{\min}$  or current depth  $\geq h_{\max}$  **then**
- 3:   Return a leaf node with center  $\mathbf{c}$  and document IDs  $\{1, \dots, m\}$
- 4: **else**
- 5:   Run spherical  $k$ -means on  $\{\mathbf{d}_i\}$  to obtain clusters  $\{\mathcal{C}_1, \dots, \mathcal{C}_k\}$
- 6:   **for**  $j = 1$  to  $k$  **do**
- 7:     Recursively build subtree  $T_j$  on vectors in  $\mathcal{C}_j$
- 8:   **end for**
- 9:   Return an internal node with center  $\mathbf{c}$  and children  $\{T_1, \dots, T_k\}$
- 10: **end if**

---

figures/hierarchical\_tree\_structure.pdf

Fig. 2: Schematic hierarchical spherical tree on the unit sphere (illustrative example with  $k=6$  branches shown for clarity). Blue markers indicate cluster centers (L1) and orange triangles show their child nodes (L2), connected via great-circle arcs. This spherical organization underlies our shallow  $k$ -ary index; in our experiments with  $k=10$  on CIFAR-100, the tree depth is 3–4 layers.

internal node’s center (L1) serves as a representative for its subtree, with child nodes (L2) forming local clusters in the vicinity of their parent on the sphere.

### C. Variance-Guided Dimension Reduction

Our scheme incorporates a variance-guided dimension reduction module as a core preprocessing component. The module can be disabled by setting  $k_{\text{ratio}}=0$  (yielding the

no-compression baseline used in our main results), or enabled with configurable compression ratios to achieve memory/communication savings. Interestingly, our experiments reveal a non-monotonic recall pattern: modest compression (e.g., 0–10% dimension removal) can sometimes *improve* retrieval recall by eliminating low-variance dimensions that carry little discriminative power for the dataset, thereby reducing noise and enhancing cluster separation on the unit sphere. Sec. ?? reports detailed trade-offs across compression ratios.

**Rule generation.** Given the document embedding matrix  $\mathbf{X} \in \mathbb{R}^{m \times n}$ , we compute per-dimension variance  $s_j = \text{Var}(\mathbf{X}_{:,j})$  and select the  $k$  smallest-variance dimensions to drop, where  $k = \lfloor n \cdot k_{\text{ratio}} \rfloor$ . Let  $\mathcal{R}$  denote the set of indices to remove and  $\mathcal{K} = [n] \setminus \mathcal{R}$  the kept indices.

**Encrypted rule.** The rule  $\mathcal{R}$  is serialized and encrypted by AES-256-CBC using a key  $K_{\text{rule}}$  generated during key setup. The ciphertext (with IV) is stored as part of the index metadata and is required at query time to ensure client/server consistency while hiding the exact rule.

**Document processing.** Each document vector is first restricted to  $\mathcal{K}$ , yielding a  $d$ -dimensional vector ( $d = n - k$ ), then L2-normalized to the unit sphere before encryption.

**Query processing.** At query time, the client decrypts the encrypted rule and applies the same keep-set  $\mathcal{K}$  to the query embedding, followed by L2-normalization and trapdoor generation. This guarantees that query and index share the same feature subspace without revealing the rule to the server.

This module is orthogonal to our hierarchical index and beam search. We report main results under  $k_{\text{ratio}} = 0$  (no compression) to focus on indexing/search; Sec. ?? (and Sec. ??) summarizes its impact succinctly.

### D. Index Encryption

For each document  $f_i$ , DO first applies the optional compression rule (if enabled) to obtain a  $d$ -dimensional vector and L2-normalizes it. Then DO pads it to length  $\ell$  by appending Gaussian noise in the last  $\ell - d$  coordinates, yielding  $\mathbf{v}_i \in \mathbb{R}^\ell$ . We split  $\mathbf{v}_i$  into two parts  $\mathbf{v}_{i,1}, \mathbf{v}_{i,2} \in \mathbb{R}^\ell$  according to  $\mathbf{s}$ :

$$\mathbf{v}_{i,1}[j], \mathbf{v}_{i,2}[j] = \begin{cases} \mathbf{v}_i[j], & \mathbf{v}_i[j], & \text{if } \mathbf{s}[j] = 0, \\ r'_{i,j}, & \mathbf{v}_i[j] - r'_{i,j}, & \text{if } \mathbf{s}[j] = 1, \end{cases} \quad (6)$$

where  $r'_{i,j}$  are fresh random numbers. Concatenating  $\mathbf{I}_i = [\mathbf{v}_{i,1}; \mathbf{v}_{i,2}] \in \mathbb{R}^{2\ell}$  and multiplying by  $\mathbf{M}^\top$  gives the encrypted index vector  $\mathbf{E}_i = \mathbf{M}^\top \mathbf{I}_i$ . For score-consistency verification, we also sample a random tag  $\mathbf{t}_i \in \mathbb{R}^{2\ell}$  and publish  $\mathbf{E}_i^{\text{auth}} = (\mathbf{t}_i - \mathbf{E}_i)/\alpha$ . Tree node centers are computed in plaintext (on normalized vectors), then encrypted in the same manner; we keep encrypted centers unnormalized in implementation.

Optionally, file contents  $f_i$  can be encrypted as  $f_i^* = \text{Enc}_{k_e}(f_i)$  using a symmetric cipher.

### E. Query Token Generation

The client decrypts the AES-encrypted compression rule (if any) to obtain the keep-set, applies the same subspace to the query embedding, L2-normalizes it, and pads to  $\ell$  with

**Algorithm 3** Beam Search on Encrypted Hierarchical Tree

**Require:** Encrypted tree  $\hat{\mathcal{T}}$ , encrypted query  $\hat{\mathbf{q}}$ , beam size  $\beta$ , top- $k$

**Ensure:** Top- $k$  encrypted document IDs and scores

- 1: Initialize beam  $\mathcal{B}_0 \leftarrow \{(\text{root}, \mathcal{S}(\hat{\mathbf{c}}_{\text{root}}, \hat{\mathbf{q}}))\}$ , level  $\ell \leftarrow 0$
- 2: **while**  $\mathcal{B}_\ell$  contains non-leaf nodes **do**
- 3:   Initialize next beam  $\mathcal{B}_{\ell+1} \leftarrow \emptyset$
- 4:   **for** each node  $u \in \mathcal{B}_\ell$  **do**
- 5:     **if**  $u$  is a leaf **then**
- 6:       Add  $u$  to  $\mathcal{B}_{\ell+1}$
- 7:     **else**
- 8:       **for** each child  $v$  of  $u$  **do**
- 9:          $s_v \leftarrow \mathcal{S}(\hat{\mathbf{c}}_v, \hat{\mathbf{q}})$ ; add  $(v, s_v)$  to  $\mathcal{B}_{\ell+1}$
- 10:      **end for**
- 11:    **end if**
- 12:   **end for**
- 13:   Prune  $\mathcal{B}_{\ell+1}$  to top- $\beta$  nodes by score;  $\ell \leftarrow \ell+1$
- 14: **end while**
- 15: Collect all document IDs from leaf nodes in  $\mathcal{B}_\ell$
- 16: For each collected document  $i$ , compute exact score  $s_i \leftarrow \mathcal{S}(\hat{\mathbf{d}}_i, \hat{\mathbf{q}})$
- 17: Return top- $k$  documents by  $s_i$

independent Gaussian noise, yielding  $\mathbf{q}_{\text{pad}} \in \mathbb{R}^\ell$ . Using the same splitting vector  $\mathbf{s}$ , we compute

$$\mathbf{q}_1[j], \mathbf{q}_2[j] = \begin{cases} r_{q,j}, \mathbf{q}_{\text{pad}}[j] - r_{q,j}, & \text{if } \mathbf{s}[j] = 0, \\ \mathbf{q}_{\text{pad}}[j], \mathbf{q}_{\text{pad}}[j], & \text{if } \mathbf{s}[j] = 1, \end{cases} \quad (7)$$

where  $r_{q,j} \sim \mathcal{N}(0, 1)$  are fresh random numbers sampled independently for each query and each coordinate. This randomized splitting (applied to coordinates where  $\mathbf{s}[j] = 0$ ) ensures that repeated queries produce computationally indistinguishable trapdoors, guaranteeing ciphertext indistinguishability and preventing correlation attacks. The splitting is complementary to index encryption: when  $\mathbf{s}[j] = 0$ , the query splits randomly while the index keeps pairs equal; when  $\mathbf{s}[j] = 1$ , the query keeps pairs equal while the index splits randomly. This duality preserves inner products. We then form  $\mathbf{T}' = [\mathbf{q}_1; \mathbf{q}_2]$ , and output the trapdoor  $\mathbf{T} = \mathbf{M}^{-\top} \mathbf{T}'$ . For verification, we also sample a random query tag  $\mathbf{r} \in \mathbb{R}^{2\ell}$  and publish  $\mathbf{T}^{\text{auth}} = (\mathbf{r} - \mathbf{T})/\alpha$  alongside the query.

With these definitions, the encrypted inner-product score satisfies  $\langle \mathbf{T}, \mathbf{E}_i \rangle = \langle \mathbf{T}', \mathbf{I}_i \rangle$ , i.e., it equals the inner product in the padded subspace.

#### F. Beam Search Algorithm

Upon receiving the encrypted query token  $\hat{\mathbf{q}}$ , CS performs beam search on the encrypted hierarchical tree  $\hat{\mathcal{T}}$  to find top- $k$  results. The key idea is to maintain  $\beta$  candidate paths at each level instead of a single greedy path.

The encrypted similarity score is computed as the inner product  $s_i = \langle \mathbf{T}, \mathbf{E}_i \rangle$ . By construction of dual transforms,  $\langle \mathbf{T}, \mathbf{E}_i \rangle = \langle \mathbf{T}', \mathbf{I}_i \rangle$ , i.e., it equals the inner product in the padded subspace defined by the common rule, ensuring correctness under encryption.

figures/merkle\_beam\_verification.png

Fig. 3: Merkle verification in a  $k=3$  hierarchical tree with depth 2 (3 layers: root, internal nodes A/B/C, leaves L1–L9). To verify that leaf L1 is included in the committed tree, the server provides a verification path (blue nodes:  $L1 \rightarrow A \rightarrow \text{Root}$ ) and sibling hashes (orange dashed boxes:  $\{L2, L3, B, C\}$ ). The client recomputes parent hashes bottom-up from L1 using the sibling values and checks that the recomputed root hash matches the published commitment  $H(\text{Root})$ . Hash formulas shown use simplified notation; full syntax with node IDs, metadata, and type tags is defined in Preliminaries.

**Complexity Analysis:** For a tree with depth  $h$  and branch factor  $k$ , beam search visits at most  $O(\beta \cdot k \cdot h)$  nodes. With  $h = \log_k n$ , the complexity is  $O(\beta k \log_k n)$ . In practice with  $k=10$  and modest  $\beta$ , the search visits only a small fraction of the corpus compared to linear scan.

#### G. Merkle-Tree-Based Integrity Verification

We augment VCSE-HST with a Merkle-tree-based integrity layer that enforces *execution integrity* for beam search: (i) all required expansions are performed; (ii) global top- $\beta$  pruning is correct; and (iii) returned documents are included in the visited leaves. The design follows a two-layer commitment, aligned with our implementation and the detailed design in our documentation.

**Index commitment.** During index construction, DO computes a bottom-up Merkle tree over the hierarchical index. For each leaf node  $u$  with encrypted center  $\hat{\mathbf{c}}_u$  and document IDs  $\text{Docs}(u)$ , we set

$$H(u) = \text{Hash}(\text{id}(u) \parallel \hat{\mathbf{c}}_u \parallel \text{Docs}(u) \parallel \text{LEAF}).$$

For each internal node with children  $v_1, \dots, v_t$  we set

$$H(u) = \text{Hash}(\text{id}(u) \parallel \hat{\mathbf{c}}_u \parallel H(v_1) \parallel \dots \parallel H(v_t) \parallel \text{INTERNAL}).$$

In our implementation we also concatenate the ordered list of child IDs to bind the structural ordering, which is compatible and strengthens the commitment. DO publishes and pins the Merkle root  $H(\text{root})$ .

**Search proofs.** For each level  $\ell$ , the server returns a level-proof containing: current beam node IDs, for every non-leaf beam node, *all* children with their scores and Merkle inclusion paths, and the claimed next-beam IDs (top- $\beta$  by score globally at that level). The client verifies: (1) inclusion of all nodes via Merkle paths w.r.t.  $H(\text{root})$ ; (2) continuity of beams across levels; (3) correctness of all scores using the score-consistency tags; and (4) correctness of pruning by recomputing the global top- $\beta$  set from the supplied expansions. Finally, the client checks that every returned document lies in the leaves of the last-level beam (via leaf proofs) and that the reported top- $k$  matches the highest verified scores among those candidates.

This enforcement guarantees integrity of the *executed* beam search (path completeness and result inclusion for the specified  $\beta$ ), while leaving the algorithmic approximation of beam search itself unchanged.

## VI. SECURITY ANALYSIS

In this section, we analyze the security of VCSE-HST. We first formalize our security model and definitions, then provide confidentiality arguments and verification mechanisms, and finally prove security theorems.

### A. Security Model and Definitions

We formalize security using the standard leakage-based paradigm for searchable encryption. Let  $\lambda$  denote the security parameter.

**Adversary model.** For *confidentiality*, we consider an honest-but-curious server observing encrypted indexes and query trapdoors. For *verifiability*, we allow a malicious server that can arbitrarily deviate from the protocol. Our scheme enforces score correctness (Theorem ??) and *execution integrity* of beam search (path completeness and result inclusion for the chosen  $\beta$ ) via Merkle commitments and level-by-level proofs.

**Leakage function.** We define  $\mathcal{L} = (\mathcal{L}_{\text{Index}}, \mathcal{L}_{\text{Search}})$ :

$$\begin{aligned}\mathcal{L}_{\text{Index}}(\mathcal{V}) &= \{m, d, \ell\}, \\ \mathcal{L}_{\text{Search}}(q, \mathcal{V}) &= \{\text{SP}(q), \text{AP}(q, \mathcal{V})\}.\end{aligned}$$

where  $m$  is the number of documents,  $d$  is the feature dimension after optional compression,  $\ell$  is the padded dimension, SP is the search pattern (repeated queries), and AP is the access pattern (returned document identifiers). These leakages align with Sec. ?? and our discussion on pattern leakage; we do not protect access patterns in this work.

**Index semantic security.** Informally, encrypted indexes  $\{\mathbf{E}_i\}$  are computationally indistinguishable from random vectors given only  $\mathcal{L}_{\text{Index}}$ . Formally, for any PPT adversary  $\mathcal{A}$  there exists a simulator  $\mathcal{S}$  such that  $\text{View}^{\text{Real}} \approx_c \text{View}^{\text{Ideal}}$  under  $\mathcal{L}$ .

**Trapdoor indistinguishability.** For distinct queries  $q_0, q_1$ , the distributions of their trapdoors are computationally indistinguishable given  $\mathcal{L}_{\text{Search}}$  (in particular, revealing only SP and AP).

**Verification soundness (score correctness).** Against a malicious server, any non-algebraic modification of reported scores is detected with overwhelming probability (Theorem ??). In addition, Merkle commitments ensure execution integrity of beam search as formalized below.

### B. Confidentiality: Simulation-Based Argument

We sketch a standard simulator proof under high-entropy padding and ASPE-style split-and-transform. Let  $\mathbf{I}_i \in \mathbb{R}^{2\ell}$  denote the split-and-pad representation of document  $i$ , and  $\mathbf{E}_i = \mathbf{M}^\top \mathbf{I}_i$  the encrypted index.

**Simulator.** Given  $\mathcal{L}_{\text{Index}}(\mathcal{V}) = \{m, d, \ell\}$ , sample  $m$  independent random vectors in  $\mathbb{R}^{2\ell}$  for  $\{\mathbf{E}_i^{\text{sim}}\}$  and independent random tags for authentication metadata; for each search, sample trapdoors consistent with  $\mathcal{L}_{\text{Search}}$  (reusing trapdoors only to reflect SP) and program outputs to match AP.

**Indistinguishability rationale.** (i) Gaussian padding of  $\ell-d$  coordinates adds high entropy; (ii) the per-coordinate splitting masks introduce randomization in complementary dimensions—for indexes, when  $s[j] = 1$ ; for trapdoors, when  $s[j] = 0$ —ensuring fresh randomness in every encrypted vector; (iii) the global invertible transform  $\mathbf{M}$  scrambles coordinates, and under high-entropy inputs the outputs are computationally close to random; (iv) the encrypted compression rule (AES-256) hides the kept-dimension set. Hence  $\text{View}^{\text{Real}} \approx_c \text{View}^{\text{Ideal}}$  under  $\mathcal{L}$ .

### C. Score Correctness Verification

We adopt a lightweight mechanism that lets the client check the *correctness of the reported inner-product scores* under encryption. Execution integrity of beam search (path completeness and result inclusion) is enforced separately by Merkle commitments (Sec. ??); we do not claim global top- $k$  optimality beyond the chosen beam algorithm. Concretely, let  $\mathbf{E}$  denote an encrypted document vector and  $\mathbf{T}$  the query trapdoor. During index construction, we additionally create an authentication vector  $\mathbf{E}^{\text{auth}}$  per document via a random secret tag  $\mathbf{t}$  and a public parameter  $\alpha > 0$ ; similarly, during trapdoor generation we create  $\mathbf{T}^{\text{auth}}$  using a query-side random tag  $\mathbf{r}$ :

$$\alpha \mathbf{E}^{\text{auth}} + \mathbf{E} = \mathbf{t}, \quad \alpha \mathbf{T}^{\text{auth}} + \mathbf{T} = \mathbf{r}. \quad (8)$$

For each returned document, the server outputs the score and two auxiliary coefficients

$$s = \langle \mathbf{T}, \mathbf{E} \rangle, \quad c_1 = \langle \mathbf{T}, \mathbf{E}^{\text{auth}} \rangle + \langle \mathbf{T}^{\text{auth}}, \mathbf{E} \rangle, \quad c_2 = \langle \mathbf{T}^{\text{auth}}, \mathbf{E}^{\text{auth}} \rangle. \quad (9)$$

Given  $(s, c_1, c_2)$  and the trapdoor-side values  $(\mathbf{T}, \mathbf{T}^{\text{auth}}, \alpha, \mathbf{r})$ , the client retrieves the per-document tag  $\mathbf{t}$  from local storage (securely distributed by DO during key distribution, never sent to CS) and checks the bilinear identity

$$\langle \mathbf{r}, \mathbf{t} \rangle = \alpha^2 c_2 + \alpha c_1 + s. \quad (10)$$

This identity detects any tampering of the inner-product score  $s$  for the returned items. This mechanism provides *score correctness*; execution integrity of the beam traversal and result inclusion is covered by Merkle proofs (Sec. ??). We do not claim global optimality beyond the beam-search approximation.

#### D. Index and Query Confidentiality

**Theorem 1:** VCSE-HST guarantees index and query confidentiality in both known ciphertext model and known background model, assuming CS does not obtain the secret keys  $(\mathbf{M}, \mathbf{s})$ .

**Proof:** CS observes encrypted document vectors  $\{\mathbf{E}_i = \mathbf{M}^\top \mathbf{I}_i\}$  and encrypted trapdoors  $\{\mathbf{T} = \mathbf{M}^{-\top} \mathbf{T}'\}$ . For the index, unknowns include the split-and-pad representations  $\{\mathbf{I}_i\}$  (of size  $2\ell$  per document, randomized by  $\mathbf{s}$  and per-coordinate masks) and the global matrix  $\mathbf{M}$  (with  $4\ell^2$  degrees of freedom). The number of linear observations from  $\{\mathbf{E}_i\}$  is insufficient to solve for both  $\{\mathbf{I}_i\}$  and  $\mathbf{M}$  jointly; the system is underdetermined. Similarly for queries, observing  $\mathbf{T} = \mathbf{M}^{-\top} \mathbf{T}'$  without  $\mathbf{M}$  and the rule does not reveal the plaintext query. Padding noise further obscures true feature values. Hence CS cannot recover plaintext vectors.

Furthermore, the dummy dimensions  $\{\varepsilon_{i,j}\}$  following a normal distribution  $\mathcal{N}(\mu, \sigma^2)$  add additional randomness that obscures the true feature values, preventing statistical attacks even in the known background model. Similar randomization techniques have been employed in data hiding [?] and secret sharing schemes [?] to enhance security.  $\square$

#### E. Token Indistinguishability

**Theorem 2:** VCSE-HST achieves query token indistinguishability for the same query request.

**Proof:** For two query tokens  $\hat{\mathbf{q}}'$  and  $\hat{\mathbf{q}}''$  generated from the same query  $q$ , the trapdoor generation process incorporates fresh randomness at two stages: (1) for coordinates where  $s[j] = 0$ , the splitting introduces per-coordinate random masks via fresh random numbers  $r_{q,j} \sim \mathcal{N}(0, 1)$ , producing independent random pairs  $(\mathbf{q}_1[j], \mathbf{q}_2[j]) = (r_{q,j}, \mathbf{q}_{\text{pad}}[j] - r_{q,j})$  that sum to the query value; (2) padding dimensions are filled with independent Gaussian noise samples.

Since both tokens use independent random samples for splitting and padding, their distributions are computationally indistinguishable even for the same underlying query. The probability that two independently generated tokens coincide is negligible in the dimension of the random space. Formally, let  $\epsilon$  denote the number of split dimensions; the collision probability is at most  $\gamma^\epsilon$  where  $\gamma \ll 1/2$ , which is negligibly small for typical values (e.g.,  $\epsilon \geq d/2$ ).  $\square$

#### F. Score Correctness and Execution Integrity

**Theorem 3** (Soundness; score correctness). Under random tags  $(\mathbf{t}, \mathbf{r})$  and secret  $\alpha$ , any modification of the reported score  $s$  (or the associated coefficients  $c_1, c_2$ ) for a returned document will, except with negligible probability, violate the identity  $\langle \mathbf{r}, \mathbf{t} \rangle = \alpha^2 c_2 + \alpha c_1 + s$ .

**Proof (Sketch):** The tags and  $\alpha$  define linear relations  $\alpha \mathbf{E}^{\text{auth}} + \mathbf{E} = \mathbf{t}$  and  $\alpha \mathbf{T}^{\text{auth}} + \mathbf{T} = \mathbf{r}$ . Expanding  $\langle \mathbf{r}, \mathbf{t} \rangle$  gives  $\alpha^2 \langle \mathbf{T}^{\text{auth}}, \mathbf{E}^{\text{auth}} \rangle + \alpha (\langle \mathbf{T}, \mathbf{E}^{\text{auth}} \rangle + \langle \mathbf{T}^{\text{auth}}, \mathbf{E} \rangle) + \langle \mathbf{T}, \mathbf{E} \rangle$ , i.e.,  $\alpha^2 c_2 + \alpha c_1 + s$ . Any change to  $(s, c_1, c_2)$  that is not algebraically consistent will be detected with overwhelming probability against a server that does not know  $(\alpha, \mathbf{t}, \mathbf{r})$ . This provides *correctness* for reported scores; we do not claim global optimality beyond beam search.  $\square$

#### Theorem 4 (Execution integrity via Merkle commitments).

Assume the hash function underlying the Merkle tree is collision resistant. Then any deviation by the server from the prescribed beam-search protocol at any level—(i) omitting children expansions for a beam node, (ii) injecting non-existent nodes, or (iii) misreporting the global top- $\beta$  pruning result—will be detected by the client with overwhelming probability using the level-by-level inclusion proofs and verified scores.

**Proof (Sketch).** For (ii), any injected node lacks a valid Merkle inclusion proof to the committed root, so verification fails by collision resistance. For (i), the client reconstructs, from the parent node's inclusion proof, the expected set of child IDs; any missing child in the reported expansions is detected by set mismatch. For (iii), the client recomputes all verified children scores at the level (using score-consistency) and deterministically recomputes the global top- $\beta$  set; any discrepancy with the server's claimed next beam is detected. The final result inclusion is verified by checking that all returned documents belong to the leaves reached by the last-level beam via leaf inclusion proofs.  $\square$

#### G. Privacy Leakage Discussion

While VCSE-HST protects index, query, and keyword privacy, it does leak some information through the tree structure and search pattern:

- **Tree structure leakage:** CS knows the hierarchical structure of the tree (which documents are in the same subtree), but not the semantic meaning of clusters due to encrypted node centers.
- **Search pattern leakage:** CS observes which tree nodes are visited during beam search, revealing information about the query's similarity pattern. Beam search visits multiple paths and thus leaks the set of explored nodes; we do not claim any privacy benefit over greedy search.
- **Access pattern leakage:** CS knows which documents are returned for each query. This is common in most SSE schemes and can be mitigated using ORAM techniques if needed.

The security-privacy trade-off can be adjusted by tuning the noise parameter  $\sigma$  in dummy dimensions and the beam size  $\beta$ . Larger  $\sigma$  provides stronger privacy but may reduce accuracy, while larger  $\beta$  visits more tree nodes but provides more search diversity.

## VII. PERFORMANCE EVALUATION

In this section, we present extensive experiments to evaluate the performance of VCSE-HST, comparing it with state-of-the-art methods including Cross-Model-SE [?] and linear search baseline.

#### A. Experimental Setup

**Dataset:** We evaluate on two large-scale datasets. (i) **CIFAR-100** [?]: we use the full training split with 50,000 images and 100 text queries (one per class). (ii) **Caltech256**:



we use the full dataset with 30,607 images and 257 text queries (one per class).

**Feature Extraction:** We use the pre-trained CLIP model (ViT-B/32) [?] to extract 512-dimensional features for both images and text queries. All features are normalized to the unit sphere.

**Implementation:** The experiments are conducted on a machine with Apple Silicon M1 processor and 16GB RAM. We implement VCSE-HST in Python 3.10 using NumPy for matrix operations.

**Parameters:**

- Tree construction: branch factor  $k = 10$ , min cluster size  $s_{\min} = 20$
- Encryption: no dimension compression ( $k_{\text{ratio}} = 0$ ); padding ratio  $\ell/d \in \{1.0, 1.05\}$ ; noise on padding  $\sigma = \sigma_q \in \{0.0, 0.01\}$
- Beam search: beam size  $\beta \in \{1, 3, 7, 10\}$
- Top- $k$  results:  $k \in \{1, 5, 10\}$

**Evaluation Metrics:**

- **Recall@K:** Fraction of queries where the top- $K$  results contain at least one document from the target class
- **Agree@1:** Fraction of queries where the top-1 result returned by VCSE-HST matches the top-1 returned by the Cross-Model-SE baseline (fidelity to baseline)
- **Search Time:** Average time per query (ms)
- **Speedup:** Ratio of baseline search time to our method's search time

**Baselines:**

- **Linear Search:** Exhaustive search over all encrypted documents
- **Cross-Model-SE:** LSH-based cross-modal searchable encryption [?] with 64 hash tables and hash dimension 128

### B. Overall Performance Comparison

Tables ?? and ?? summarize performance on the full CIFAR-100 (50,000 images) and Caltech256 (30,607 images) datasets. Note that the recall rates reflect the CLIP model's retrieval capability, not limitations of the encryption schemes. Thus, accuracy metrics are only compared among CLIP-based schemes (Cross-Model-SE, Linear Search, and VCSE-HST). Cross-Model-SE with tuned parameters (64 tables, 128-dim hashes) reaches 90% R@1 on CIFAR-100 and serves as the latency baseline (57.5ms/query). VCSE-HST with beam size 10 matches 90% R@1 while being 7.5 $\times$  faster (7.66ms/query); with beam size 3, it yields up to 19.7 $\times$  speedup (2.91ms/query) with only 7% R@1 loss. On Caltech256, beam size 10 similarly achieves 7.5 $\times$  speedup at comparable accuracy.

We also compare with MU-TEIR [?] and Pitr [?] in terms of computational overhead. Since these schemes use different feature extraction models, only time and memory metrics are compared (accuracy comparison would be meaningless). Detailed performance metrics are shown in Table ?. The optional compression module is evaluated separately in Sec. ?.

**Key Observations:**

- 1) **Beam size provides flexible trade-offs:** By adjusting  $\beta$  from 1 to 10, we can smoothly trade accuracy for speed.

figures/paper\_accuracy\_comparison.png

Fig. 4: Accuracy comparison (Recall@1, Recall@5, Recall@10) for different schemes. VCSE-HST with beam size 10 matches Cross-Model-SE's accuracy while being much faster. Even beam size 3 achieves competitive Recall@5 and Recall@10.

Beam size 3 offers the best cost-performance ratio (83% R@1, 19.7 $\times$  speedup), while beam size 10 achieves state-of-the-art accuracy with significant speedup.

- 2) **Shallow tree structure is crucial:** Our 3–4 level hierarchical tree enables both fast search and high accuracy, avoiding the accumulated errors of deeper trees.
- 3) **Superior to LSH-based methods:** VCSE-HST with beam size 10 matches Cross-Model-SE's 90% Recall@1 while being 7.5 $\times$  faster, demonstrating the effectiveness of hierarchical tree indexing combined with beam search.

Fig. ?? visualizes the accuracy comparison across different recall metrics.

### C. Search Time Analysis

Fig. ?? compares average search time on a log scale; VCSE-HST attains 7.5–19.7 $\times$  speedups over Cross-Model-SE under our settings.

The millisecond-level search time of VCSE-HST is attributed to: (1) a shallow tree structure (3–4 levels), (2) efficient beam pruning that visits only a small fraction of nodes, and (3) optimized inner product computations on normalized vectors.

### D. Beam Size Trade-off Analysis

We report accuracy and search time as beam size  $\beta$  varies (no compression, no noise). Fig. ?? and Fig. ?? present the two perspectives separately for clearer layout.

**Observations:**

TABLE I: Performance comparison on CIFAR-100 (50,000 images, 100 queries)

| Scheme                    | R@1 (%)   | R@5 (%) | R@10 (%) | Agree@1 (%) | Search Time (ms) | Speedup <sup>†</sup> (vs. Cross-Model-SE) | Index Size (MB) |
|---------------------------|-----------|---------|----------|-------------|------------------|-------------------------------------------|-----------------|
| Cross-Model-SE            | 90        | 97      | 98       | 100         | 57.47            | 1.0×                                      | 518.89          |
| Linear Search             | 90        | 97      | 98       | 100         | 144.44           | 0.4×                                      | 782.10          |
| VCSE-HST ( $\beta = 3$ )  | 83        | 96      | 97       | 70          | 2.91             | <b>19.7×</b>                              | 782.10          |
| VCSE-HST ( $\beta = 7$ )  | 89        | 97      | 98       | 86          | 6.39             | <b>9.0×</b>                               | 782.10          |
| VCSE-HST ( $\beta = 10$ ) | <b>90</b> | 97      | 99       | 90          | 7.66             | <b>7.5×</b>                               | 782.10          |
| MU-TEIR*                  | —         | —       | —        | —           | 12,570           | 0.005×                                    | ~60             |
| PITR*                     | —         | —       | —        | —           | 64               | 0.90×                                     | 28.5            |

<sup>†</sup>Speedup is calculated relative to Cross-Model-SE (avg. per-query time).

\*Accuracy metrics not comparable due to different feature extraction models.

TABLE II: Performance on Caltech256 (full: 30,607 images, 257 queries)

| Scheme                    | R@1 (%)     | R@5 (%) | R@10 (%) | Agree@1 (%) | Search Time (ms) | Speedup <sup>†</sup> (vs. Cross-Model-SE) | Index Size (MB) |
|---------------------------|-------------|---------|----------|-------------|------------------|-------------------------------------------|-----------------|
| Cross-Model-SE            | 88.7        | 96.5    | 98.1     | 100         | 34.38            | 1.0×                                      | 317.79          |
| Linear Search             | 88.7        | 96.5    | 98.1     | 100         | 86.12            | 0.4×                                      | 478.70          |
| VCSE-HST ( $\beta = 10$ ) | <b>88.7</b> | 96.5    | 98.1     | 90          | 4.58             | <b>7.5×</b>                               | 478.70          |

<sup>†</sup>Speedup is calculated relative to Cross-Model-SE (avg. per-query time).

figures/paper\_search\_time\_comparison.png

Fig. 5: Search time comparison on logarithmic scale. VCSE-HST attains 7.5–19.7× speedup over Cross-Model-SE under our settings. Beam size 10 achieves single-digit millisecond latency.

- **Diminishing returns:** Increasing beam size improves agreement (e.g., 70% → 86% → 90% for  $\beta = 3, 7, 10$ ), but the marginal gain diminishes at larger beams.
- **Sub-linear time growth:** Search time grows sub-linearly with beam size due to early termination when leaf nodes are reached.
- **Sweet spots:** Beam size 3 for high-throughput scenarios (19.7× speedup with moderate agreement), and beam size 10 for fidelity-critical scenarios (high agreement, still

figures/paper\_beam\_accuracy.png

Fig. 6: Top-1 agreement rate increases with beam size under no-compression setting.

7.5× faster).

#### E. Accuracy–Speed Trade-off

Fig. ?? summarizes top-1 agreement (bars, left axis) and average search time (line, right log axis) in one view, highlighting the agreement–efficiency balance achieved by different schemes.

#### F. Noise Robustness (No-Compression Baseline)

We evaluate robustness under lightweight padding noise on top of the no-compression baseline. Concretely, we add 5%

figures/paper\_beam\_time.png

Fig. 7: Search time increases sub-linearly with beam size (log-scale) under no-compression setting.

figures/paper\_acc\_time\_combo.png

Fig. 8: Agreement-speed trade-off across schemes. Larger beam sizes improve agreement while maintaining millisecond-level latency.

padding to the vector length ( $\ell = 1.05d$ ) and inject small Gaussian noise ( $\sigma = \sigma_q = 0.01$ ) into padding dimensions, leaving original dimensions intact. Our experiments show that (i) lightweight padding noise preserves accuracy—R@1 remains stable within  $\pm 2\%$  across Linear and HST beam sizes; (ii) time overhead is negligible (0–5%) for HST thanks to sub-linear traversal and fixed tree structure; (iii) separating padding from original dimensions enables security tunability without degrading retrieval quality.

figures/paper\_compression\_accuracy.png

Fig. 9: Non-monotonic relationship between compression ratio and Recall@1 on CIFAR-100. Modest compression (2.5–5%) improves recall by eliminating low-discriminative dimensions, peaking at 93% R@1 with 5% compression. Beyond 10%, recall degrades as informative dimensions are pruned.

#### G. Compression Ratio Study and Analysis

To quantify the impact of dimension reduction alone, we vary the compression ratio  $k_{\text{ratio}}$  under *no noise* and *beam=10* on CIFAR-100. As shown in Fig. ??, we observe a striking *non-monotonic* recall pattern: starting from no compression, recall initially *rises* to about 92% at 2.5% compression and peaks at about 93% R@1 with 5% compression, then declines to 90% at 7.5%, 88% at 10%, and 86% at 12.5% compression. Beyond 10%, recall continues to degrade (84% at 15%, 82% at 20%) as informative dimensions are pruned. This counterintuitive improvement at modest compression ratios demonstrates that removing low-variance dimensions can enhance retrieval by reducing noise, though the non-monotonic pattern suggests complex interactions between signal preservation and noise reduction.

**Theoretical interpretation.** We explain this trend using a signal-noise decomposition. Let a normalized vector decompose as  $\mathbf{d} = \mathbf{s}_d + \boldsymbol{\eta}_d$ , where  $\mathbf{s}_d$  lies in a signal subspace and  $\boldsymbol{\eta}_d$  aggregates weak/irrelevant components. Cosine similarity admits a first-order approximation

$$\cos(\mathbf{q}, \mathbf{d}) \approx \frac{\langle \mathbf{s}_q, \mathbf{s}_d \rangle + \langle \boldsymbol{\eta}_q, \boldsymbol{\eta}_d \rangle}{\sqrt{(\|\mathbf{s}_q\|^2 + \|\boldsymbol{\eta}_q\|^2)(\|\mathbf{s}_d\|^2 + \|\boldsymbol{\eta}_d\|^2)}}.$$

Removing weak-variance coordinates reduces the denominator and the variance of spurious terms  $\langle \boldsymbol{\eta}_q, \boldsymbol{\eta}_d \rangle$ , improving the separation between true and distractor similarities. On the hierarchical spherical tree, this decreases center-angle jitter and the probability of branching mistakes in beam search. When compression exceeds a threshold and starts pruning

TABLE III: Index Construction Metrics on CIFAR-100 (50,000 images)

| Scheme         | Construction Time (s) | Index Size (MB) |
|----------------|-----------------------|-----------------|
| Cross-Model-SE | 10.27                 | 518.89          |
| PITR           | 11.50                 | 28.5            |
| Linear Search  | 7.73                  | 782.10          |
| VCSE-HST       | 10.08                 | 782.10          |

informative coordinates, the numerator  $\langle s_q, s_d \rangle$  diminishes and recall degrades. This explains the observed “initial rise then fall” without contradicting the information-loss intuition.

#### H. Index Construction Time

Table ?? compares index construction time and memory overhead on the full CIFAR-100 dataset (50,000 images).

VCSE-HST’s index construction time (10.08s) is comparable to Cross-Model-SE (10.27s) and faster than PITR (11.50s), while being slightly higher than Linear Search (7.73s) due to the additional overhead of hierarchical tree construction and Merkle commitment computation. PITR’s longer construction time reflects the cost of building HNSW’s multi-layer graph structure. The index size is larger than Cross-Model-SE (782MB vs 519MB) because we store full encrypted vectors rather than compact hash codes, though still significantly larger than PITR’s compressed representation (28.5MB). However, the memory overhead is acceptable for modern systems, and the significant search speedup (7.5–19.7 $\times$  over Cross-Model-SE) justifies this trade-off.

#### I. Integrity Verification Overhead

We evaluate the cost of Merkle-tree-based integrity verification on CIFAR-100 using our reports with  $\beta \in \{1, 3, 5, 7, 10\}$ . Fig. ?? shows that average per-query verification time scales near-linearly with beam size and remains in the few-millisecond range. Fig. ?? decomposes total latency into search and verification components.

Finally, Fig. ?? summarizes the average index construction overhead introduced by adding Merkle commitments; the extra hashing is modest compared to overall preprocessing.

**Limitation.** The Merkle layer guarantees execution integrity with respect to the chosen beam size  $\beta$  (path completeness and result inclusion along the executed search). It does not guarantee global top- $k$  completeness beyond the beam-search exploration. The verification cost grows approximately linearly with  $\beta$ .

### VIII. CONCLUSION

In this paper, we proposed VCSE-HST, a verifiable cross-modal searchable encryption scheme based on a hierarchical spherical tree with beam search. By constructing a shallow  $k$ -ary tree and employing beam search to avoid local optima, our scheme attains strong accuracy–efficiency trade-offs: on CIFAR-100 (50,000) and Caltech256 (30,607), beam size 10 matches Cross-Model-SE (90% R@1) while being 7.5 $\times$  faster; beam size 3 yields up to 19.7 $\times$  speedup with 7% R@1 drop.

figures/paper\_merkle\_verification\_time\_vs\_beam.pdf

Fig. 10: Verification time vs beam size for Merkle-based integrity checks. Overhead grows near-linearly with  $\beta$  and stays within a few milliseconds per query.

figures/paper\_merkle\_time\_breakdown.png

Fig. 11: Per-query latency breakdown (search + verification). Verification constitutes a small fraction of total time and does not change accuracy metrics.

We enable *score-consistency verification* under encryption: clients check the correctness of returned scores using lightweight auxiliary coefficients, and we enforce *execution integrity* of beam search via Merkle commitments. We do not claim global optimality beyond the beam-search approximation. Security analysis shows that VCSE-HST protects



figures/paper\_merkle\_index\_time.png

Fig. 12: Index construction time: Cross-Model-SE vs VCSE-HST + Merkle (ms). Merkle hashing adds modest overhead at build time.

index, query, and keyword privacy under honest-but-curious adversaries.

**Limitation.** Our scheme does not hide access/search patterns: the server can observe returned identifiers and the set of visited nodes. Beam search explores multiple paths and may reveal a larger visited-node set than greedy search; protecting patterns requires orthogonal techniques (e.g., ORAM, DPF), which we leave to future work.

Future work includes: (1) extending the scheme to support dynamic updates with forward and backward security, (2) exploring adaptive beam size strategies based on query characteristics, (3) optimizing index compression to reduce memory overhead, and (4) evaluating scalability on larger datasets (e.g., 100K+ documents). We also plan to investigate hybrid index structures combining hierarchical trees with locality-sensitive hashing for even better performance on ultra-large-scale datasets.

#### ACKNOWLEDGMENT

The author would like to thank the anonymous reviewers for their valuable comments and suggestions.